

AuthLog

1.1 Introduzione

Il progetto consiste nella realizzazione di un sistema IoT per il controllo degli accessi basato su tecnologia RFID, capace di verificare in tempo reale l'identità di un utente attraverso un database remoto e di registrare automaticamente un log di ogni operazione. Il sistema è inoltre in grado di comandare l'apertura di una serratura, consentendo l'accesso esclusivamente agli utenti autorizzati.

La soluzione integra diverse tecnologie: un microcontrollore STM32 dedicato alla gestione dell'hardware e della lettura RFID, un modulo Wi-Fi ESP8266 per la comunicazione remota, il servizio cloud Supabase per la gestione e l'archiviazione dei dati e un'app mobile sviluppata in React Native che permette la registrazione degli utenti e l'accesso ai servizi tramite login. A queste componenti si aggiunge un sensore di distanza basato su tecnologia Time-of-Flight (ToF), utilizzato per rilevare la presenza dell'utente e migliorare l'interazione con il sistema.

Il progetto non solo risponde alle esigenze dei moderni sistemi di controllo accessi, ma rappresenta anche un esempio concreto di integrazione tra più aree tecnologiche: elettronica digitale, programmazione embedded, comunicazione seriale, reti e protocolli Internet, servizi cloud e sviluppo mobile. Tutte le componenti sono state progettate e realizzate in autonomia, affrontando problemi reali come la comunicazione tra microcontrollori, la gestione sicura delle API, la sincronizzazione dei dati e l'usabilità dell'interfaccia utente.

L'integrazione del sensore di prossimità consente di rendere il sistema più intelligente e interattivo, permettendo di attivare il dispositivo solo in presenza dell'utente e fornendo un feedback visivo immediato tramite LED. Questo approccio migliora l'affidabilità del sistema e riduce il rischio di attivazioni involontarie.

Si tratta inoltre di una soluzione facilmente applicabile e di grande interesse negli attuali scenari IoT, dove sono richiesti elevati standard di sicurezza, affidabilità e usabilità per la gestione degli accessi in laboratori, uffici o aree riservate al personale autorizzato.

1.2 Problema di partenza

Negli ultimi anni l'evoluzione dei sistemi di sicurezza ha portato a un crescente utilizzo di tecnologie IoT per il controllo degli accessi in ambienti professionali, laboratori, strutture scolastiche e aree riservate al personale autorizzato. I tradizionali sistemi basati su chiavi fisiche o badge offline risultano spesso limitati, difficili da aggiornare e poco sicuri, poiché non consentono una gestione dinamica degli accessi né un controllo remoto delle attività.

Il progetto nasce dall'esigenza di realizzare una soluzione moderna, flessibile e facilmente integrabile che permetta:

- la **verifica immediata delle credenziali** di un utente tramite un database remoto costantemente aggiornabile;
- la **registrazione automatica dei log**, utile sia per motivi di sicurezza sia per la tracciabilità delle operazioni, permettendo di rilevare quando l'utente ha fatto l'accesso;
- la **gestione centralizzata degli utenti**, tramite un'app mobile intuitiva, che permette di avere un accesso veloce e sempre disponibile da parte di tutti gli utenti ;
- la possibilità di **comandare una serratura elettronica**, consentendo l'accesso solo agli utenti effettivamente autorizzati;
- un sistema **scalabile**, replicabile e compatibile con diversi ambienti IoT.

Questa esigenza è particolarmente rilevante in contesti scolastici e professionali, dove il controllo degli accessi deve essere affidabile, tracciabile e semplice da aggiornare senza interventi manuali sul dispositivo.

1.3 Obiettivi del progetto

Il progetto ha come obiettivo principale la realizzazione di un sistema IoT completo per il controllo degli accessi, capace di identificare un utente tramite RFID, verificarne l'autorizzazione attraverso un database remoto e registrare automaticamente ogni tentativo di accesso.

L'idea è quella di creare un dispositivo che possa essere utilizzato in contesti reali, come laboratori scolastici, aree riservate o ambienti industriali, dove è necessario un elevato livello di affidabilità e un controllo costante delle persone abilitate.

Per raggiungere questo risultato, il sistema integra più componenti: un microcontrollore STM32 per la gestione della parte elettronica, un modulo Wi-Fi ESP8266 per la comunicazione con il cloud, e un servizio online (Supabase) utilizzato sia come database sia come gestore delle API. L'app mobile sviluppata in React Native completa il sistema, permettendo la registrazione e il login degli utenti e rendendo così il progetto facilmente utilizzabile anche da un pubblico non tecnico. Inoltre, è stato integrato un sensore di distanza basato su tecnologia Time-of-Flight (ToF), utilizzato per rilevare la presenza dell'utente e migliorare l'interazione con il sistema.

Un ulteriore obiettivo è stato quello di garantire che ogni accesso venga registrato automaticamente, inserendo nel database un log contenente data, ora e codice RFID letto. Questo permette di avere uno storico preciso e consultabile degli ingressi. Il sensore di prossimità contribuisce inoltre a rendere il sistema più intuitivo, fornendo un feedback visivo tramite LED e permettendo di attivare il dispositivo solo quando l'utente è effettivamente presente.

Dal punto di vista tecnico e formativo, il progetto mira anche a esplorare l'interazione tra firmware embedded, comunicazione seriale, connessione HTTPS verso servizi cloud e sviluppo di applicazioni mobile. L'obiettivo è stato quindi non solo realizzare un sistema funzionante, ma costruire un progetto completo e scalabile, capace di integrare competenze provenienti da più ambiti tecnologici.

1.4 Tecnologie utilizzate

Il progetto integra diverse tecnologie hardware e software, che cooperano per realizzare un sistema di controllo accessi completamente funzionante e connesso al cloud.

La parte elettronica è basata su un microcontrollore **STM32 Nucleo-64 F401RE**, scelto per la sua affidabilità, la disponibilità di porte di comunicazione e la compatibilità con l'ambiente Arduino, che ha semplificato lo sviluppo del firmware. A questo è stato affiancato un modulo **ESP8266 (ESP-12E)**, utilizzato per la connessione Wi-Fi e la gestione delle comunicazioni HTTPS con il backend remoto.

La comunicazione tra STM32 ed ESP8266 avviene tramite interfaccia seriale dedicata, mentre l'identificazione dell'utente è gestita tramite tecnologia **RFID** (NFC passivo), che fornisce un codice univoco associabile a un singolo accesso. A completamento del sistema è stata introdotta una componente per il rilevamento della distanza, basata su tecnologia Time-of-Flight (ToF). In particolare è stata utilizzata la scheda X-NUCLEO-53L4A2

Per la parte cloud è stato utilizzato **Supabase**, una piattaforma che offre un database PostgreSQL completamente gestito e accessibile tramite API REST. Supabase è stato impiegato per memorizzare gli utenti registrati, gli utenti autorizzati e lo storico degli accessi, consentendo un controllo centralizzato e facilmente scalabile del sistema. La comunicazione tra ESP8266 e Supabase avviene tramite richieste HTTPS, con autenticazione tramite chiavi API.

La componente mobile è stata sviluppata con **React Native**, che ha permesso di realizzare un'applicazione compatibile con dispositivi Android e iOS. L'app consente agli utenti di registrarsi, effettuare il login e interagire con il sistema in modo semplice, sfruttando il database di Supabase tramite il client ufficiale. Sono state utilizzate librerie per la navigazione tra schermate e per la gestione dell'interfaccia grafica, con l'obiettivo di rendere l'app intuitiva e utilizzabile anche da utenti non tecnici.

La scelta delle tecnologie è guidata da alcuni requisiti fondamentali del progetto:

- *sicurezza dei dati*, garantita dall'uso di HTTPS e database Supabase;
- *bassa latenza*, ottenuta grazie alla comunicazione Wi-Fi diretta tra ESP8266 e server;
- *semplicità di manutenzione*, resa possibile dall'aggiornabilità del firmware e dalle API REST;
- *affidabilità dei componenti*, assicurata da hardware consolidato come STM32 e ESP8266.

Un elemento particolarmente innovativo del progetto è l'impiego dello smartphone come dispositivo NFC per l'autenticazione, eliminando la necessità di badge o tag fisici. Questa scelta tecnologica offre vantaggi significativi: lo smartphone, infatti, è un dispositivo che

l'utente possiede e porta sempre con sé, riducendo i costi e le complessità legate alla distribuzione e gestione dei tag RFID tradizionali. L'NFC integrato nei telefoni moderni garantisce inoltre una comunicazione rapida e affidabile, mentre la presenza di blocco schermo, funzioni biometriche e sistemi operativi sicuri aggiunge un ulteriore livello di protezione rispetto ai tag fisici, che possono essere copiati o smarriti.

2 Architettura del Sistema

2.1 Architettura hardware

L'architettura può essere rappresentata attraverso il flusso logico immagine 2.1.1:

RFID

Il tag RFID emette un codice univoco quando avvicinato al lettore.

STM32

Raccoglie il codice RFID e gestisce la logica locale e il rilevamento della distanza, comandando anche l'apertura della serratura.

Trasferisce il codice all'ESP8266 tramite seriale.

ESP8266

Si occupa della comunicazione con Internet e della verifica dell'utente interrogando Supabase.

Riceve comandi e restituisce allo STM32 l'esito dell'autorizzazione.

Supabase (cloud)

Contiene il database delle autorizzazioni, degli utenti registrati e dei log. Risponde alle richieste della ESP8266 tramite API REST.

App Mobile

Interagisce direttamente con Supabase per la registrazione e il login degli utenti.

Opera indipendentemente dalla rete locale del dispositivo.

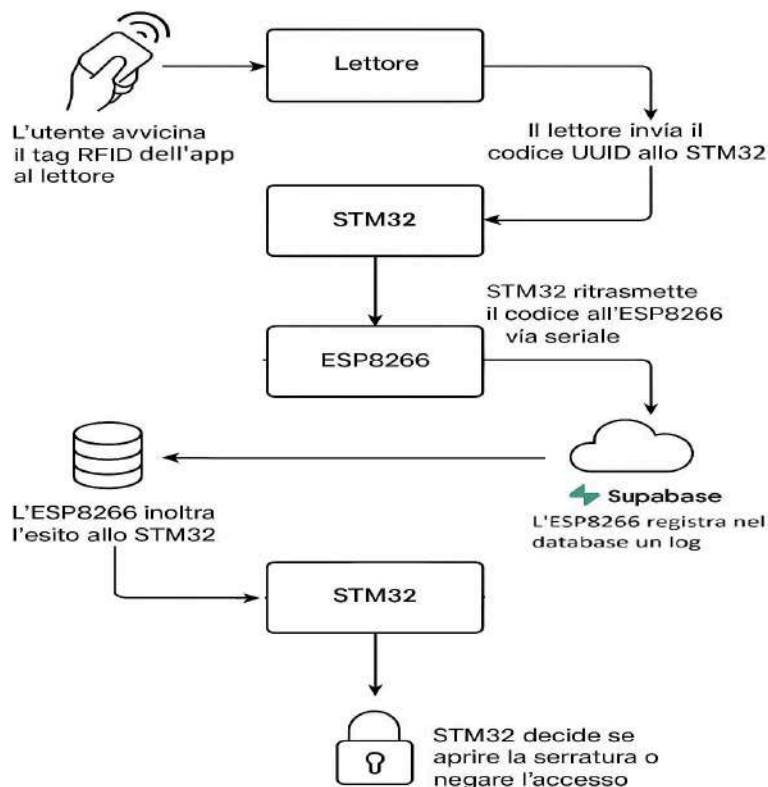


immagine 2.1.1

2.2 Protocollo di comunicazione Nucleo64 e ESP8266

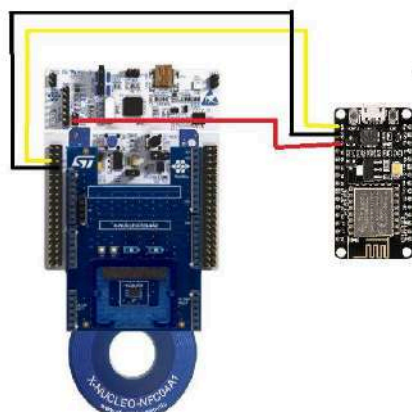
La comunicazione tra le due componenti hardware, Nucleo64 e ESP8266, avviene tramite una comunicazione **UART** (Universal Asynchronous Receiver-Transmitter), un'interfaccia seriale ampiamente supportata sia dalla Nucleo64 che dal ESP8266.

Le sue principali caratteristiche nel nostro sistema sono:

- **Velocità (baud rate):** 115200 baud.
- **Formato dati:** 8N1 (8 bit dati, nessuna parità, 1 bit di stop).

Il collegamento fisico utilizza:

- Nucleo64 TX → ESP8266 RX
- GND in comune
- Alimentazione 3.3V in comune



ROSSO: TX → RX
NERO: GND
GIALLO: 3.3V

immagine 2.2.1

La Nucleo 64 comunica l'UUID ricevuto alla ESP8266, con questo formato:

UUID:<codice>

es:

UUID:d1013489-8e93-40bf-81bc-98ee74ddd700

Una volta ricevuto l'UUID la ESP8266 risponderà con "OK" se la richiesta al server è avvenuta con successo, e quindi se esiste un utente con quel UUID associato, e ipoteticamente invierebbe un segnale elettrico per aprire la porta. Altrimenti risponde con "NO" e non invia nessun segnale alla porta.

```
if (checkUUIDExists(uuid)) {  
  Serial.println("UUID TROVATO → OK");  
  
  // REGISTRA LOG  
  sendLogToSupabase(uuid);  
  
  Serial.println("LOG inviato.");  
  Serial.println("OK");  
}
```

Immagine 2.2.2

2.3 Comunicazione REST API con Supabase

Per garantire affidabilità, sicurezza e compatibilità con un microcontroller a risorse limitate, è stato adottato un modello di comunicazione basato su API REST e protocollo HTTPS, forniti nativamente da Supabase.

L'ESP8266 comunica con Supabase utilizzando il protocollo **TLS 1.2**, necessario per stabilire una connessione sicura prima dello scambio dei dati.

L'uso di HTTPS garantisce la protezione da:

- intercettazioni del traffico (man-in-the-middle),
- manipolazione dei dati inviati o ricevuti,
- furto delle credenziali o dei codici RFID.

Le chiamate API verso Supabase seguono un formato standard e avvengono tramite la libreria `HTTPClient` dell'ESP8266.

Ogni richiesta contiene tre elementi fondamentali:

- **URL dell'endpoint REST:** <https://<project>.supabase.co/rest/v1/Authorized>
- **Headers di autenticazione:** "apikey", "Authorization: Bearer <API_KEY>"
- **Filtri e parametri della query:** [?uuid=eq.<valore>](https://<project>.supabase.co/rest/v1/Authorized?uuid=eq.<valore>)

Quando il modulo ESP8266 riceve uno UUID dalla STM32, invia una richiesta GET per verificare se quel codice è presente nella tabella Authorized.

es: GET <https://<project>.supabase.co/rest/v1/Authorized?uuid=eq.<codice>>

Supabase risponde sempre con un array JSON.

Ogni tentativo di accesso viene registrato in Supabase nella tabella Logs. La registrazione avviene tramite una chiamata POST che invia un oggetto JSON contenente:

- l'UUID del tag RFID
- la data e l'ora dell'operazione

```
{
  "uuid": "d1013489-8e93-40bf-81bc-98ee74ddd700",
  "log_time": "2025-11-24T14:52:00Z"
}
```

esempio di log

immagine 2.3.1

3 Hardware Utilizzato

3.1 Nucleo-64 STM32F401RE

La scheda STM32 Nucleo-64 è il microcontrollore centrale del sistema. Svolge la gestione diretta dell'hardware locale, come il lettore RFID e gli eventuali dispositivi di output (relè per la serratura). È stato scelto per vari motivi:

- è affidabile e largamente utilizzato in contesti didattici e industriali;
- possiede una buona disponibilità di pin digitali e UART dedicate;
- può essere programmato anche tramite IDE Arduino, semplificando lo sviluppo;

La Nucleo-64 si occupa:

- di leggere l'**UUID dal lettore RFID**;
- comunica con l'**ESP8266** tramite interfaccia seriale **UART**;
- decidere, in base alla risposta ricevuta, se abilitare o meno l'**apertura della serratura**;



3.2 Modulo Wi-Fi ESP8266 (ESP-12E)

L'ESP8266 è il componente responsabile della connessione del sistema a Internet. Nello specifico, è stato utilizzato il modulo **ESP-12E**, una versione avanzata del classico ESP8266 dotata di più pin e migliore affidabilità.

Il modulo svolge funzioni critiche:

- ricezione del codice RFID via seriale dallo STM32;
- comunicazione con Supabase tramite protocolli HTTPS;
- invio di richieste REST per la verifica degli accessi;
- registrazione dei log nella tabella remota;
- risposta allo STM32 con esito "OK" o "NO".



La sua funzione principale consiste nel ricevere tramite comunicazione seriale il codice RFID proveniente dallo STM32 e utilizzarlo per interrogare Supabase attraverso richieste HTTPS. Dopo aver verificato l'esistenza o meno dell'utente all'interno della tabella delle autorizzazioni, il modulo invia una risposta semplice e immediata allo STM32, comunicando l'esito con un "OK" in caso di accesso consentito o "NO" in caso contrario. Parallelamente, l'ESP8266 si occupa anche di registrare nel database un log contenente il codice letto e il momento esatto della transazione, permettendo così un tracciamento completo degli accessi.

Inoltre, è un modulo pienamente compatibile con le comuni reti Wi-Fi a 2.4 GHz, facilmente integrabile grazie al supporto offerto dalle librerie Arduino dedicate e dotato di una vasta documentazione utile nelle fasi di sviluppo e debug.

3.3 Lettore RFID STM32 NFC04A1

Il lettore RFID costituisce il punto di interazione fisica tra l'utente e il sistema.

A ogni avvicinamento di un tag da cellulare, il lettore lo riconosce immediatamente e ne estrae il codice univoco (UUID), che funge da identificatore dell'utente.



La scelta del lettore RFID è stata guidata da criteri di affidabilità e compatibilità: si tratta infatti di un dispositivo capace di lavorare con tag ampiamente diffusi in ambito scolastico e lavorativo, garantendo letture rapide e precise anche a breve distanza.

3.4 Expansion Board ToF 53L4A2

La scheda X-NUCLEO-53L4A2 è un modulo basato su tecnologia Time-of-Flight (ToF), utilizzato per misurare la distanza tra il dispositivo e l'utente. È basata sul sensore VL53L4CD e comunica con la STM32 tramite interfaccia I2C.

Il sensore permette di rilevare la presenza dell'utente in prossimità del sistema, migliorando l'interazione e rendendo possibile l'attivazione di segnali visivi tramite LED in base alla distanza rilevata.



4 Software

Nella seguente sezione viene illustrato le parti fondamentali del codice utilizzato per far funzionare il progetto.

4.1 Nucleo-64 F401RE + RFID NFC04A1

Per permettere la lettura NFC è stata utilizzata una libreria apposita per la scheda, nell'IDE di Arduino.

La comunicazione seriale è stata configurata con parametri standard (baud rate costante e terminatore di riga) per assicurare una trasmissione affidabile e facilmente interpretabile dal modulo Wi-Fi.

La funzione principale del firmware è quella di ricevere dal lettore RFID il codice identificativo (UUID) associato al tag presentato dall'utente. Se la lettura non va a buon fine viene stampato un codice errore.

Se la lettura va a buon fine, una volta letto il codice, il microcontrollore lo elabora in forma testuale e lo trasmette al modulo ESP8266 attraverso l'interfaccia UART dedicata.

4.2 ESP8266

Il firmware installato sul modulo ESP8266 costituisce la parte più complessa e strategica del sistema software, poiché si occupa della gestione della connettività Wi-Fi, della comunicazione con il database remoto e dell'elaborazione dei dati ricevuti dal microcontrollore STM32. È il componente che trasforma un semplice sistema elettronico locale in una soluzione IoT completa, capace di verificare autorizzazioni in tempo reale e sincronizzare informazioni verso un servizio cloud.

In primo luogo, l'ESP8266 si connette alla rete Wi-Fi utilizzando le credenziali configurate nel codice.

Una volta stabilita la connessione, il modulo resta in ascolto sulla porta seriale per ricevere i messaggi inviati dallo STM32. Ogni volta che arriva un codice RFID, il modulo esegue una fase preliminare di validazione per assicurarsi che il messaggio sia coerente e formattato correttamente.

```
#include "ST25DVSensor.h"
```

```
void setup() {  
  SerialDebug.begin(115200); // debug USB  
  SerialESP.begin(115200);   // UART verso ESP8266  
  delay(500);  
  
  if(st25dv.begin() == 0) {  
    SerialDebug.println("System Init done!");  
  } else {  
    SerialDebug.println("System Init failed!");  
    while(1);  
  }  
}
```

```
// Legge l'UUID / stringa scritta dal telefono  
if(st25dv.readURI(&uuidRead)) {  
  SerialDebug.println("Read failed!");  
  delay(500);  
  return;  
}
```

```
if(uuidRead.length() > 0) {  
  // Rimuove https:// se presente  
  if(uuidRead.startsWith("https://")) {  
    uuidRead = uuidRead.substring(8);  
  } else if(uuidRead.startsWith("http://")) {  
    uuidRead = uuidRead.substring(7);  
  }  
}
```

```
// Invia l'UUID via UART alla ESP8266  
SerialESP.println(uuidRead);
```

```
#define WIFI_SSID  
#define WIFI_PASSWORD
```

```
void loop() {  
  
  if (Serial.available()) {  
    String uuid = Serial.readStringUntil('\n');  
    uuid.trim();  
  }  
}
```

Dopo la validazione, l'ESP8266 avvia una richiesta HTTPS verso Supabase. Questa fase è particolarmente delicata, poiché richiede la gestione di una connessione sicura tramite TLS e l'invio di headers specifici contenenti le chiavi API necessarie per l'autenticazione. L'ESP8266 utilizza le API REST di Supabase per interrogare la tabella degli utenti autorizzati e verificare se il codice comunicato dallo STM32 corrisponde a un accesso consentito.

```
String url = TABLE_AUTH_URL + "?uuid=req." + uuid;

Serial.println("Richiesta a: " + url);

if (!http.begin(client, url)) {
    Serial.println("Errore begin()");
    return false;
}

http.addHeader("apikey", SUPABASE_API_KEY);
http.addHeader("Authorization", "Bearer " + SUPABASE_API_KEY);
http.addHeader("Content-Type", "application/json");

int code = http.GET();
Serial.print("HTTP code: ");
Serial.println(code);
```

Una volta ottenuta la risposta dal server, il firmware analizza il contenuto del JSON ricevuto per determinare l'esito della verifica.

Oltre alla verifica dell'accesso, il firmware della ESP8266 svolge anche la funzione di registrazione dei log. Viene inviata una seconda richiesta HTTP verso Supabase, questa volta di tipo POST, contenente il codice letto e il timestamp dell'evento. Questa operazione consente di mantenere uno storico completo degli accessi, utile sia a fini di monitoraggio che di sicurezza.

```
// JSON per Supabase
String json = "{\"uuid_auth\": \"" + uuid + "\"}";
// NOTA: log_time viene generato automaticamente da Supabase

Serial.println("Invio log: " + json);

int code = http.POST(json);
Serial.print("POST LOG → HTTP code: ");
Serial.println(code);

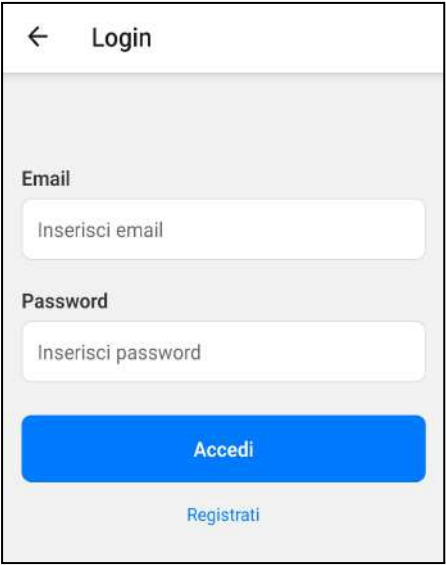
if (code > 0) {
    String response = http.getString();
    Serial.println("Risposta log:");
    Serial.println(response);
}

http.end();
```

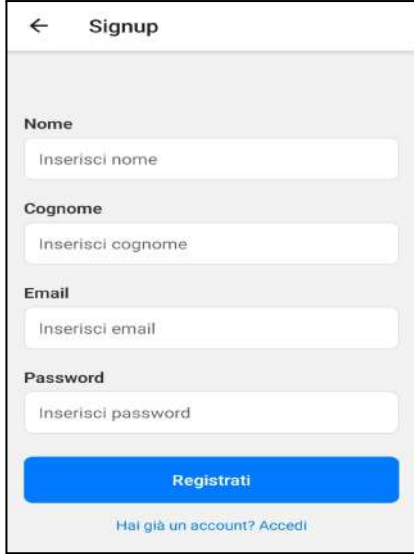
4.3 App Mobile (React Native)

L'applicazione mobile rappresenta l'interfaccia destinata all'utente finale e costituisce un elemento essenziale del sistema, poiché consente la gestione degli account, la registrazione di nuovi utenti e l'accesso ai servizi collegati al sistema di controllo accessi. È stata sviluppata in React Native, una tecnologia che permette la creazione di applicazioni multiplatforma utilizzando JavaScript e componenti nativi, garantendo un'esperienza fluida sia su Android che su iOS. L'architettura dell'app è basata su due schermate principali:

- La schermata di registrazione: consente all'utente di creare un nuovo profilo compilando i campi necessari (nome, cognome, email e password) dati che vengono immediatamente inviati a Supabase tramite una chiamata POST per essere inseriti nella tabella *User*.



The Login screen features a back arrow and the title 'Login'. It contains two input fields: 'Email' with the placeholder 'Inserisci email' and 'Password' with the placeholder 'Inserisci password'. Below the fields is a large blue button labeled 'Accedi'. At the bottom, there is a link 'Registrati' in blue text.



The Signup screen features a back arrow and the title 'Signup'. It contains four input fields: 'Nome' (placeholder 'Inserisci nome'), 'Cognome' (placeholder 'Inserisci cognome'), 'Email' (placeholder 'Inserisci email'), and 'Password' (placeholder 'Inserisci password'). Below the fields is a large blue button labeled 'Registrati'. At the bottom, there is a link 'Hai già un account? Accedi' in blue text.

- La schermata di login: verifica invece la presenza dell'utente nella tabella *Authorized*, che contiene le credenziali generate o validate dall'amministratore del sistema.

La comunicazione con Supabase avviene attraverso il client ufficiale per JavaScript, integrato nell'app tramite la libreria `@supabase/supabase-js`. L'utilizzo di questo client permette di effettuare tutte le operazioni necessarie tramite semplici funzioni, mantenendo il codice leggibile e riducendo la complessità delle chiamate HTTP. Supabase funge da backend completo: gestisce le tabelle degli utenti, le credenziali di accesso e i token di autenticazione, evitando così la necessità di implementare un server dedicato.

La terza pagina dell'app mobile è la HomePage visualizzabile dopo essersi autenticati tramite la pagina di Login. La pagina visualizza l'email dell'utente che ha fatto l'accesso. Inoltre comunica all'utente se l'NFC è attivo nel dispositivo mobile, visualizzando un messaggio di allerta.



The Home screen features a back arrow and the title 'Home'. It displays a welcome message 'Benvenuto!' followed by the email 'prova@gmail.com'. Below the email, there is a green checkmark and the text 'NFC rilevato e pronto'. A green button labeled 'Scrivi UUID su Tag RFID' is positioned above a red button labeled 'Logout'.



The Home screen features a back arrow and the title 'Home'. It displays a welcome message 'Benvenuto!' followed by the email 'prova@gmail.com'. Below the email, there is an orange warning message: 'NFC potrebbe non essere attivo. Puoi provare comunque a scrivere'. A green button labeled 'Scrivi UUID su Tag RFID' is positioned above a red button labeled 'Logout'.

Cliccando sul pulsante verde “Scrivi UUID su Tag RFID” viene scritto l’UUID del utente Authorized nel NFC del telefono, pronto per essere scritto sulla scheda NFC04A1.

```
<TouchableOpacity
  style={[styles.nfcButton, isWriting && styles.buttonDisabled]}
  onPress={() => {
    try {
      handleWriteNfc();
    } catch (error: any) {
```

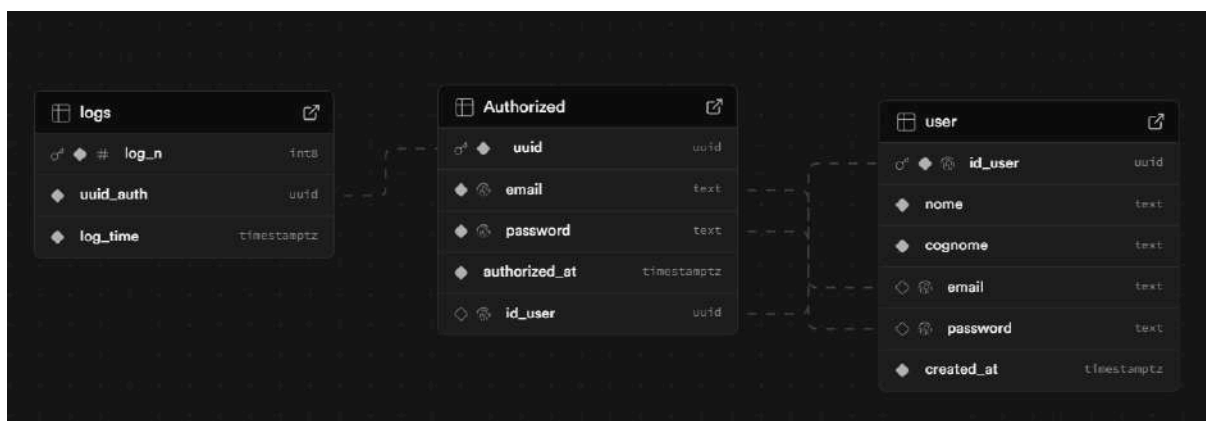
Questo processo è eseguito tramite la funzione `handleWriteNfc()` che si occupa di inizializzare l’NFC e controllare eventuali errori.

La navigazione tra le pagine .tsx è gestita tramite il file `App.tsx`, che utilizza la libreria `@react-navigation/native` per permettere ciò.

```
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Login">
        <Stack.Screen name="Login" component={LoginScreen} />
        <Stack.Screen name="Signup" component={SignupScreen} />
        <Stack.Screen name="Home" component={HomeScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

4.4 Database Supabase

Supabase rappresenta il backend centrale del sistema ed è la componente che permette di gestire in modo strutturato e sicuro i dati relativi agli utenti, ai codici RFID e ai log degli accessi. La scelta di questo servizio è stata motivata dalla necessità di avere una piattaforma affidabile, veloce da integrare e completa di funzionalità come database, autenticazione, API REST automatiche e gestione dei permessi, senza dover sviluppare un’infrastruttura server personalizzata.



Il cuore del backend è costituito da un database **PostgreSQL**, all'interno del quale sono state definite tre tabelle principali: *User*, dedicata alla registrazione degli utenti tramite app; *Authorized*, che contiene le credenziali necessarie per l'accesso al sistema; e *Logs*, dove vengono registrati gli eventi generati dall'ESP8266 quando un codice RFID viene presentato. Questa struttura permette una separazione chiara tra l'identità dell'utente, la sua autorizzazione effettiva e la traccia delle operazioni, consentendo un controllo accurato e sicuro dei flussi di accesso.

Gli utenti registrati tramite app vengono registrati nella tabella *user*, ma non possono fare l'accesso nelle zone autorizzate, perché non è stato ancora associato un UUID per fare l'accesso. Solo l'amministratore può autorizzare un user. Quando autorizzato verrà registrato un nuovo record che abbia come primary key l'UUID associato, e erediterà gli attributi email e password dalla tabella *user*. Ogni volta che un utente *Authorized* farà accesso alla zona autorizzata verrà registrato un log che abbia come foreign key l'UUID *Authorized* e la data e ora di quando è avvenuto l'accesso.

Nella tabella *User* tutti gli attributi tranne: nome, cognome, *created_at*; Sono attributi unique, ovvero può esistere solo un record con lo stesso patrimonio informativo.

Per garantire la sicurezza del sistema, Supabase utilizza le Row Level Security (RLS), politiche che permettono di stabilire quali utenti o quali componenti possano accedere o modificare i dati. Nel progetto sono state definite policy che impediscono agli utenti non autenticati di leggere informazioni sensibili e che autorizzano solo l'ESP8266, tramite chiave di servizio, a consultare la tabella *Authorized* e a inserire nuovi record nella tabella *Logs*.

In questo modo, anche qualora qualcuno ottenesse l'URL dell'endpoint API, non avrebbe comunque i permessi necessari per accedere ai dati.

La comunicazione tra dispositivi e backend avviene tramite REST API generate automaticamente da Supabase. Ogni tabella può essere raggiunta attraverso un endpoint dedicato, sul quale è possibile eseguire operazioni di SELECT, INSERT, UPDATE o DELETE semplicemente tramite chiamate HTTP. Il microcontrollore ESP8266, ad esempio, effettua una richiesta GET filtrata sull'UUID per verificare la presenza di un utente autorizzato, mentre una richiesta POST viene utilizzata per inserire un nuovo log.

Per la comunicazione con il backend viene utilizzata la "service key" di Supabase, un token con privilegi elevati pensato per l'uso lato server. Questa chiave permette al dispositivo di bypassare l'autenticazione classica e di accedere direttamente alle tabelle necessarie. Nel progetto è stata conservata in chiaro solo durante la fase di sviluppo: in un'applicazione reale sarebbe opportuno proteggerla tramite meccanismi hardware, firmware cifrato o proxy intermedi.

5 Conclusioni e sviluppi futuri

Il progetto ha permesso di realizzare un sistema completo per il controllo degli accessi basato su tecnologia RFID e integrazione IoT, riuscendo a combinare in un'unica soluzione hardware, firmware, backend e applicazione mobile.

L'obiettivo iniziale, creare un sistema capace di identificare un utente tramite tag RFID,

verificarne l'autorizzazione su un database remoto e registrare un log dell'operazione è stato pienamente raggiunto.

Durante lo sviluppo sono state affrontate problematiche di natura molto diversa: dalla comunicazione seriale tra microcontrollori alla gestione delle API REST, dalla configurazione delle politiche di sicurezza del backend alla sincronizzazione con l'applicazione mobile. Ogni parte del progetto ha richiesto competenze specifiche e l'integrazione finale ha dimostrato l'efficacia dell'approccio modulare adottato.

Uno dei risultati più significativi è stato ottenere un sistema realmente funzionante e facilmente estendibile, che può essere adattato a diversi contesti come accessi a laboratori, magazzini o ambienti aziendali, ma anche utilizzato come base per soluzioni domotiche avanzate.